

CareerHub — Profile Data Schema

The single source of truth behind the resume builder, ATS checker, and autofill engine

Every CareerHub tool reads from and writes to one structured Profile. This document defines that schema — every field, its validation rules, and how the resume builder, ATS checker, and autofill engine each consume the same data differently. Getting this right once means every future tool plugs into existing data instead of asking the user to re-enter anything.

1. Why a Single Shared Schema

The entire product promise — "enter your details once, use them everywhere" — only holds if every tool is built against the same underlying data model rather than each tool having its own local form. The Profile is the product's real asset; resume templates, ATS scoring, and autofill are three different views onto it.

- **Resume Builder** renders selected Profile sections into a chosen visual template.
- **ATS Checker** compares Profile content (skills, experience text) against a job description's keywords.
- **Autofill Engine** maps individual Profile fields to matched form fields on external portals.
- **Cover Letter Generator** and future tools (mock interview prep, application tracker) all read the same Profile rather than requesting data again.

2. Schema Structure — Top-Level Sections

Section	Type	Contains	Consumed by
personal	Object	Name, contact info, address, photo, links.	Resume header, autofill contact fields.
education	Array of objects	One entry per degree/qualification, ordered most recent first.	Resume education section, autofill cascading dropdowns.
experience	Array of objects	One entry per job/internship.	Resume experience section, ATS keyword matching, autofill employment history.

Section	Type	Contains	Consumed by
skills	Array of strings + proficiency tags	Flat list with optional proficiency level.	Resume skills section, ATS keyword matching, autofill 'key skills' tag fields.
projects	Array of objects	Optional -- especially relevant for students/freshers with limited work experience.	Resume projects section.
certifications	Array of objects	Name, issuer, date, credential ID/URL.	Resume section, some portals' certification fields.
documents	Object	References to uploaded resume file(s), photo, signature -- not raw data, just storage pointers.	Resume export, autofill file-upload fields, exam-form photo/signature specs.
preferences	Object	Job preferences: desired role, location, salary expectation, notice period, work mode.	Autofill preference dropdowns (salary bucket, notice period), job-match filtering (future feature).
sensitiveIds	Object, opt-in only	Aadhaar/PAN/passport-style values -- not created by default.	Autofill only when explicitly enabled per-use; never included in resume/export.

3. Field-Level Detail & Validation Rules

personal

Field	Type	Validation	Notes
fullName	string	Required, 2-100 chars, letters/spaces/periods only	Resume header, autofill name fields (also split into firstName/lastName on demand).
email	string	Required, standard email regex validation	Resume header, universal autofill target -- highest-confidence field across all portals.
phone	string	Required, stored as digits only + country code separately	Autofill formats per-portal (with/without country code, with/without spaces) at fill-time, not stored pre-formatted.
dob	date (ISO 8601)	Required for exam-form use cases; optional for resume	Stored once, reformatted per-portal at fill-time (DD/MM/YYYY vs MM/DD/YYYY vs separate day/month/year fields).
address	object (line1, line2, city, state, pincode, country)	Pincode validated against country-specific pattern (6 digits for India)	Autofill address blocks; resume header (city/state only, not full address, by default for privacy).
photo	file reference	Enforced dimensions/size per use-case (see Documents section)	Exam-form photo upload, optional resume photo (off by default -- many ATS systems penalize photos on resumes).
links	array (LinkedIn, portfolio, GitHub, etc.)	URL format validation per link type	Resume header, autofill 'LinkedIn URL' / 'portfolio' fields.

education (array item)

Field	Type	Validation	Notes
degree	string (from controlled list + 'Other')	Required; controlled list matches common portal dropdown options directly	Reduces autofill fuzzy-matching errors on Naukri/Foundit-style cascading dropdowns.
institution	string	Required, 2-150 chars	Free text -- fuzzy matched against portal's institution search-select where applicable.
board/university	string	Required for school-level entries (SSC/board exams relevant to govt job forms)	Specifically needed for SSC/IBPS-style forms that ask board name separately from institution.
yearOfCompletion	integer (year)	Required, range-validated (1980-current year+1)	Autofill year dropdowns; resume date formatting.

Field	Type	Validation	Notes
percentage/ CGPA	number + type flag	Required, range-validated based on type (0-100 or 0-10 scale)	Some govt exam forms require percentage even if CGPA was awarded -- store both if user provides a conversion.

experience (array item)

Field	Type	Validation	Notes
jobTitle	string	Required, 2-100 chars	Resume, ATS keyword source, autofill 'designation' fields.
company	string	Required	Resume, autofill 'employer name' fields.
startDate / endDate	date (ISO 8601), endDate nullable	startDate required; null endDate = 'Present'	Reformatted per-portal at fill-time; 'Present' flag drives portal-specific 'currently working here' checkboxes.
description	rich text / bullet array	Required, min 1 bullet, recommend action-verb + quantified format	Primary ATS keyword-matching source; resume body content.
employmentT ype	enum (Full-time, Internship, Contract, Freelance)	Required	Autofill employment-type dropdowns.

4. Documents Sub-Schema (Photo/Signature Specs)

Government/bank exam forms are notoriously strict about photo and signature specs, and rejections for wrong file size/dimensions are common and avoidable. The Documents section stores the source image once and generates compliant variants on demand rather than asking the user to re-upload per portal.

Field	Contains	Validation
photo.master	Original uploaded photo, highest resolution available	Face-visible check (basic), min resolution threshold
photo.variants	Auto-generated resized/compressed versions per known spec (e.g. IBPS: 200x230px, 20-50KB; SSC: specific separate spec)	Each variant validated against its target spec before being offered for use
signature.master	Original uploaded signature scan	Basic contrast/crop check
signature.variants	Auto-generated resized versions per known exam-body spec	Same per-spec validation as photo
resume.files	Array of generated resume exports (PDF/DOCX), one per template used	Each tagged with template ID + generation date

This turns a common, frustrating failure point (wrong photo size causing form rejection) into a one-time setup that pays off across every future exam/job application -- a strong retention hook specific to the Indian competitive-exam audience.

5. How Each Tool Reads the Same Schema Differently

```
Profile (single source of truth)
|
|-- Resume Builder    -> selects sections + template -> renders to PDF/DOCX
|                       (user can hide sections per-resume, e.g. hide projects
|                       for an experienced-hire resume, without editing Profile data)
|
|-- ATS Checker       -> extracts skills[] + experience[].description text
|                       -> compares against job description keywords
|                       -> does NOT modify Profile; read-only consumer
|
|-- Autofill Engine   -> flat-maps every leaf field (personal.phone, education[0].degree, etc.)
|                       -> matches against detected form fields via scoring engine
|                       -> read-only consumer; never writes back to Profile automatically
|
|-- Cover Letter Gen  -> reads experience[] + skills[] + target job description
|                       -> generates draft text; user edits before use
|
|-- Application Tracker -> references Profile by ID per application record,
|                       does not duplicate Profile data into each tracked application
```

Key architectural rule: only the Profile editor writes to the Profile. Every other tool is a read-only consumer. This prevents one tool's assumptions (e.g. an AI rewrite) from silently corrupting the source data other tools depend on.

6. Handling Multiple Resume Variants from One Profile

Users commonly need slightly different resumes for different roles (e.g. emphasizing different skills for a backend vs. frontend role). Rather than duplicating the whole Profile, each saved resume stores a lightweight "view config" on top of the shared Profile:

```
resumeVariant = {
  id: "resume_backend_v1",
  profileRef: "profile_main",      // always points back to the single Profile
  template: "minimal-ats-safe",
  sectionOrder: ["experience", "skills", "projects", "education"],
  skillsFilter: ["Node.js", "PostgreSQL", "System Design"], // subset shown, not a data copy
  experienceOverrides: {
    "exp_002": { descriptionOverride: "...tailored bullet text for this variant only..." }
  }
}
```

The **experienceOverrides** pattern is important: it lets a user tailor wording for one variant without touching the master data that ATS-checker and autofill rely on for accuracy elsewhere.

7. Data Lifecycle & Consent Rules

- Sensitive ID fields (sensitiveIds.*) are never created by default -- the user must explicitly opt in per field, and each is excluded from autofill by default even once stored (per the Autofill Spec's prohibitions).
- Photo variants are regenerated on demand, not stored indefinitely for every possible spec -- reduces stored data footprint to only specs the user has actually used.
- Deleting a Profile field does not retroactively edit already-exported resume files -- exports are immutable snapshots, only the live Profile is mutable.
- Full Profile export (JSON) and full delete are both single actions available to the user at any time, matching the data-control commitments in the Autofill Engine Spec.
- Profile edit history is kept minimally (last-modified timestamp per field) -- not a full audit log -- to avoid retaining more personal data than necessary.

With this schema in place, every future tool (mock interview prep, application tracker, exam-form assistant) is a new "view" on existing data rather than a new data-entry burden on the user -- which is the actual mechanism behind the product's core promise.